APPENDIX A
ARTIFACT APPENDIX

This artifact accompanies *Isotaxis: Optimal Asynchronous Byzantine Consensus with Ordering Linearizability*. It provides the Isotaxis and Pompe implementations, a common HotStuff backend, four-replica Docker experiments, the global EC2 workflow, and the data and script for Figs. 2, 3, and 5–7. These components evaluate the latency, throughput–latency, and ordering-layer denial-of-service (DoS) results in Section VIII of the paper. Operational details are in **`README-AE.md`**.

A. Description & Requirements

The main Rust crate contains **Isotaxis**, **Pompe**, **clients**, and **adversarial nodes**; the bundled second crate is their modified HotStuff backend. The two Compose files run the normal and attack cases. Global deployment scripts are under `ec2/`, and `gen-fig.py` regenerates the plots.

1) How to access: The repository is available at https://github.com/xrxrxrxrxr/Isotaxis-ArtifactEvaluation. Clone the evaluated revision:

```
git clone \
  https://github.com/xrxrxrxrxr/\
Isotaxis-ArtifactEvaluation
cd Isotaxis-ArtifactEvaluation
```

The evaluated commit is recorded in `README-AE.md`; the camera-ready appendix will cite the AEC-approved archive DOI. The authors' code is Apache-2.0; third-party components retain their original licenses.

2) Hardware dependencies: The local four-replica run needs an x86-64 host with 8 logical cores, 16 GiB RAM, and 20 GiB free disk; no GPU is needed. CPU-affinity settings refer to cores 0–3. Docker Desktop can run the functional test, but its measurements are not directly comparable to the paper. A full rerun needs one dedicated EC2 `c6a.2xlarge` (8 vCPUs, 16 GiB) per process and a `t3.micro` client, distributed over 11 regions for up to 100 processes.

3) Software dependencies: Local experiments require Git, Docker Engine, and Docker Compose; testing used Engine 28.3.2, Compose v2.38.2, and Buildx v0.25.0. Rust is built inside the image. Plotting needs Python 3.9+ and `matplotlib==3.9.4`, pinned in `requirements-plot.txt`. E3 additionally needs AWS CLI v2, `jq`, SSH/SCP, GNU Make, credentials, and the EC2 fleet above.

4) Benchmarks: No dataset is required: the client generates synthetic transactions. `ORDERING_BATCH_SIZE` is the nominal transactions per ordering input; the paper uses one for sparse latency and sweeps up to 1.6×10^5 for throughput. Processed series for Figs. 2, 3, and 5–7 are embedded in `gen-fig.py`; live runs produce client and per-replica logs.

B. Artifact Installation & Configuration

From the repository root, verify the revision and both Compose configurations:

```
git rev-parse HEAD
docker compose config --quiet
docker compose --env-file .attack.env \
  -f docker-compose.attack.yml config -q
```

The revision must match `README-AE.md`, and both configuration checks must succeed. E1 builds Rust inside Docker. For E2:

```
python3 -m venv .venv-ae
source .venv-ae/bin/activate
python -m pip install -r \
  requirements-plot.txt
```

C. Experiment Workflow

E0 validates the package; E1.1 and E1.2 run the local normal and DoS paths; E2 renders the archived paper data; and optional E3 collects new global EC2 data. Because `run_test.sh` clears `logs`, preserve each run first. For comparisons, use at least three repetitions with the same commit, environment, hardware, batch size, and attack rate, then compare medians.

D. Major Claims

- **(C1) Sparse-load scalability.** The confirmation latency of Pompe grows faster with replica count. At $n = 100$, Fig. 5 reports about 22 s for Pompe and 12 s for Isotaxis. E1.1 checks the path; E2 checks the archived result.
- **(C2) Throughput–latency trade-off.** Batching initially raises throughput until resource limits; Isotaxis's advantage grows with n. At $n = 100$ and batch size $\approx 1.3\times10^4$, it exceeds 28,000 transactions/s, nearly twice Pompe. Figs. 3 and 6 are checked by E2 and optionally rerun by E3.
- **(C3) Robustness to ordering-layer DoS.** With one of 30 processes flooding, Fig. 7 shows Isotaxis changing from 2.597 to 3.067 s and from 1,155 to 978 transactions/s; Pompe changes from 2.030 to 33.474 s and from 1,478 to 90 transactions/s. E1.2 checks the path and E2 the data.

The paper's safety, liveness, resilience, and asymptotic communication claims are established analytically and are not treated as artifact-evaluation claims.

E. Evaluation

1) Experiment (E0): package and build validation: [5–20 human-minutes plus first-build time]. Preparation/Execution: run the installation checks above. *Results:* all commands exit zero and the revision matches; E1 must subsequently build four replica containers and the load generator.

2) Experiment (E1.1): local normal execution: [About 5 human-minutes and 4 compute-minutes per run]. Preparation: in `.env`, set `CLIENT_ORDERING_MODE=smrol` and `ORDERING_BATCH_SIZE=1` for the four-node sparse-latency path. Use a larger batch for a throughput check. *Execution:*

```
./run_test.sh --rebuild
./run_test.sh --reuse
```

Repeat with `CLIENT_ORDERING_MODE=pompe` for the baseline. Preserve the logs before switching. *Results:* `logs/load_test.log` contains aggregate ordering/consensus latency, and `logs/node/consensus-node*.log` shows continuing commits and throughput. This is the four-node counterpart of Figs. 3, 5, and 6, not their global scale.

3) Experiment (E1.2): local DoS execution: [About 5 human-minutes and 4 compute-minutes per run]. Preparation: preserve matching E1.1 logs, then set in `.attack.env`:

```
CLIENT_ORDERING_MODE=smrol
ADVERSARY_MODE=smrol
CLIENT_EXCLUDED_NODE_IDS=1
ATTACK_SLEEP_MICROS=100
ATTACK_START_DELAY_MS=2000
```

For Pompe, change both protocol variables to `pompe` and set `POMPE_ATTACK_WORKERS=4`. *Execution:*

```
./run_test.sh adversary --rebuild
./run_test.sh adversary --reuse
```

Results: node1 reports the pure attacker and HotStuff participation; all replicas keep committing and the client prints its aggregate report. Pompe also prints `[pompe-attacker][rate]`. Across matched repetitions, Pompe's median ordering latency should degrade markedly, while Isotaxis remains comparatively stable, matching Fig. 7 qualitatively. Illustrative four-node logs are in `log-examples/attack-logs`; new results are in `logs`.

4) Experiment (E2): regenerate the paper figures: [About 2 human-minutes; less than 1 compute-minute]. Preparation/Execution:

```
source .venv-ae/bin/activate
mkdir -p python-fig
python gen-fig.py
```

Results: ten PDFs appear under path `python-fig`: those figures are the `attacks-intro`, `tradeoff-100-intro`, and `latency` PDFs map to Figs. 2, 3, and 5; the six `tps-*/tradeoff-*` PDFs map to Fig. 6; and `attacks-exp.pdf` maps to Fig. 7. They must show the C1–C3 values above. E2 renders archived processed series; it is not a new AWS measurement.

5) Experiment (E3): global AWS execution for Fig. 6: [Optional; about 5 compute-minutes per point or 4.5 compute-hours for the full sweep]. This costly, state-changing workflow selects existing instances. Each region must have the security-group rules documented in README-AE.md; account-specific identifiers are not included in the artifact. First inspect the plan and dry-run:

```
cd ec2
NODE_INSTANCE_TYPE=c6a.2xlarge \
  ./start_instances.sh \
  --plan plans/plan-16.json --dry-run
```

Plans 16, 64, and 100 correspond to the Fig. 6 panels. On an authorized account with existing instances and their SSH private key, *execute:*

```
export SSH_KEY_PATH=/path/to/key.pem
./start_instances.sh \
  --plan plans/plan-16.json
NODE_IP_SOURCE=public ./get_ips.sh
make init && make deploy
make pull && make start
```

These steps generate configuration, initialize and deploy the hosts, pull the pinned Docker Hub image, run the experiment, stop the containers, and collect logs. Repeat the documented batch/load sweep for both protocols and record the plan and configuration. *Results:* `ec2/ec2-logs` contains client latency and committed throughput for reconstructing the Fig. 6 panel.

F. Customization

Use `.env` for normal runs and `.attack.env` for attacks. Main controls are the protocol, ordering batch size, excluded client targets, attack interval, and Pompe worker count. An `ec2/plans/plan-x.json` selects the distributed replica count and regions. Record any CPU-affinity changes.